Feature Engineering

Índice

Introducción 01

¿Qué es feature engineering? 02

Feature Engineering: 03

- **Gestión de omitidos**
- **Categoricas**
- **De tiempo**
- **Numéricas**

¿Qué es el Feature Engineering?

Definición:

Es el proceso de crear nuevas variables o modificar las existentes con el fin de mejorar el rendimiento de los modelos de ML. Involucra extraer información significativa de datos en bruto, reducir el ruido y transformar variables para hacerlas más adecuadas para el modelo.

El feature engineering se hace por las siguientes razones:

- Mejorar la precisión del modelo
- Reducir el overfitting o sobreajuste del modelo
- Mejorar la explicabilidad

Tipos de variables: recordatorio

Tipo de Variable	Definición	Ejemplos
Numérica	Variables que representan valores cuantitativos y pueden ser medidas aritméticamente.	Edad, Ingresos, Temperatura
Categórica	Variables que representan categorías o etiquetas y no tienen un orden intrínseco.	Género, Color, Tipo de Producto
Texto	Variables que consisten en datos textuales que pueden contener oraciones o párrafos.	Comentarios, Descripciones, Direcciones
Temporal	Variables que representan instantes o períodos de tiempo.	Fecha de Nacimiento, Hora, Año
Booleanas	Variables que solo pueden tomar dos valores posibles: verdadero (True) o falso (False).	Sí/No, Verdadero/Falso, 1/0
Ordinales	Variables categóricas que tienen un orden natural o jerarquía.	Nivel educativo (Primaria, Secundaria, Universidad), Niveles de satisfacción
Geoespaciales	Variables que representan ubicaciones geográficas.	Coordenadas GPS, Direcciones, Regiones

Librerías Python

- Scikit-Learn
- Pandas
- Numpy
- Matplot lib
- Seaborn
- NLTK

Tipo de feature engineering

Feature engineering

Numéricas

- Scaling and Normalization
- Mathematical Transformations (e.g., logarithm)
- Discretización
- Binarización

Categóricas

- One-Hot Encoding
- Label Encoding
- Target Encoding

Gestión de Omitidos

- Eliminar variables
- Eliminar registros omitidos
- imputación

Texto

- Bag of Words (BoW)
- TF-IDF (Term Frequency-Inverse Document Frequency)
- Word Embeddings (e.g., Word2Vec)

De tiempo

- Extracting Date Components (Year, Month, Day)
- Creating Time Lags

No hay por qué hacer uso de todas ellas

Gestión de omitidos

No todos los algoritmos de Machine Learning que vamos a aplicar admiten la presencia de valores nulos u omitidos, por lo que hay que definir estrategias para gestionar estos valores nulos.

Se llama **data cleaning,** es el proceso de limpieza de datos que hay que llevar a cabo para que pueda ser input del algoritmo de ML.

1. **Eliminar los registros a nulo**
2. **Eliminar la variable completa**
3. **Reemplazar por un valor (0, media, mediana, moda, ...)**

Gestión de omitidos

1. **Eliminar los registros a nulo**

   df = df.dropna(subset=[‘variable’])

2. **Eliminar la variable completa**

   df = df.drop([‘variable1’, ‘variable2’, ‘variable3’], axis=1)

3. **Reemplazar por un valor (0, media, mediana, moda, ...)**

   median=df[‘variable’].median()
   df['variable1'].fillna(median, inplace=True)

Variables categóricas

Generalmente las variables categóricas no son admitidas como input en un algoritmo de ML por lo que es necesario convertirlas en algo numérico.

1. **One Hot Encoder**
2. **Label encoding**
3. **Target encoding**

Variables categóricas: One Hot Encoder

¿Qué es One-Hot Encoding?

El One-Hot Encoding es una técnica utilizada en el procesamiento de datos para convertir variables categóricas en un formato que los algoritmos de ML puedan entender. Básicamente, cada categoría única se convierte en una nueva columna binaria (0 ó 1), donde 1 indica la presencia de esa categoría y 0 su ausencia.

1. **Con Panda:**

```
df_encoded = pd.get_dummies(df, columns=['variable'])
```

2. **Con Scikit Learn:**

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False)
encoded_data = encoder.fit_transform(df[[variable]])
```

Variables categóricas: Label encoding

Label encoding, también conocido como integer encoding, es una técnica que asigna un número entero único a cada categoría \ de una variable categórica. Este método es simple pero puede introducir una relación ordinal que no existe, lo que puede ser problemático para ciertos modelos de aprendizaje automático.

Ejemplo:

Si tienes una variable categórica "Tipo de Vivienda" con categorías 'Estudio', 'Apartamento', 'Loft' y 'Duplex', estas categorías se codifican como 1, 2, 3 y 4, respectivamente.

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()

data['Tipo de Vivienda Codificado'] = label_encoder.fit_transform(data['Tipo de Vivienda'])
```

El orden que aplica es alfabético

```
from sklearn.preprocessing import OrdinalEncoder

orden_categorias = ['Estudio', 'Apartamento', 'Loft', 'Duplex']

ordinal_encoder = OrdinalEncoder(categories=[orden_categorias])

data['Tipo de Vivienda Codificado'] = ordinal_encoder.fit_transform(data[['Tipo de Vivienda']])
```

Con Ordinal_encoder asignamos un orden a las categorías

Variables categóricas: Target encoding

Target encoding, también conocido como mean encoding, es una técnica de codificación categórica que transforma variables categóricas en valores numéricos basados en la variable objetivo. En lugar de asignar un número aleatorio a cada categoría, se utiliza la media de la variable objetivo para cada categoría. Esto puede mejorar el rendimiento del modelo al capturar la relación entre las categorías y la variable objetivo.

Ejemplo:

Si tienes una variable categórica "Color" con categorías 'Rojo', 'Azul' y 'Verde', y tu variable objetivo es 1 para 'éxito' y 0 para 'fracaso', cada color se reemplaza por el promedio de la variable objetivo para todas las instancias de ese color

```
# Calcular el Target Encoding
target_means = data.groupby('Categoria')['Objetivo'].mean()
data['Categoria Codificada'] = data['Categoria'].map(target_means)
```

Usamos map para reemplazar valores en un dataframe

Variables de tiempo

Esta técnica consiste en extraer de una variable tiempo el día, mes, año, hora, minuto, etc. Es útil cuando hay eventos que queremos predecir que tienen cierta estacionalidad.

```
import pandas as pd
data = pd.DataFrame({
    'Fecha': ['2025-02-04 05:59:00', '2024-06-12 14:22:00', '2023-11-23 09:15:00']
})

# Convertir la columna 'Fecha' a un objeto datetime
data['Fecha'] = pd.to_datetime(data['Fecha'])

# Extraer el año, mes, día, hora y minuto
data['Año'] = data['Fecha'].dt.year
data['Mes'] = data['Fecha'].dt.month
data['Día'] = data['Fecha'].dt.day
data['Hora'] = data['Fecha'].dt.hour
data['Minuto'] = data['Fecha'].dt.minute
```

Variables numéricas: Scaling

Scalation (Escalado): El escalado de características es el proceso de ajustar los valores de las características para que estén en una misma escala o rango. Hay algoritmos de ML que pueden hacer que variables en diferentes escalas puedan condicionar el resultado del modelo. Por ejemplo si estamos pensando en hacer un modelo donde utilizamos información de nuestros clientes, como la edad (unas decenas) o la renta media del lugar donde vive van a tener unos órdenes de magnitud diferentes,

Hay diferentes métodos de escalado, dos de los más comunes son:

1

Min-Max Scaling

Transforma los valores a un rango específico, normalmente [0, 1].:

$$Xscaled = \frac{X - X\min}{X\min - X\max}$$

```
min_max_scaler = MinMaxScaler()
data_min_max_scaled = min_max_scaler.fit_transform(data)
```

2

Standard Scaling (Estandarización):

Ajusta los valores para que tengan una media de 0 y una desviación estándar de 1.

$$Xscaled = \frac{X - \mu}{\sigma}$$

```
standard_scaler = StandardScaler()
data_standard_scaled = standard_scaler.fit_transform(data)
```

Variables numéricas: Normalización

Normalizacion. La normalización también ajusta los valores, pero el objetivo es hacer que las características tengan magnitudes similares sin necesidad de estar en el mismo rango. Se utilizan diferentes técnicas dependiendo del contexto, las dos más comunes son:

1

Normalización L2:

Escala cada vector de características (fila de datos) para que su norma L2 sea 1. Esto significa que la suma de los cuadrados de los valores será 1:

$$X_{nomralized} = \frac{X}{\|X\|_2}$$

$$\|X\|_2 = \sqrt{X_1^2 + X_2^2 + \cdots + X_n^2}$$

```
normalizer = Normalizer(norm='l2')
data_normalized = normalizer.fit_transform(data)
```

2

Normalización de rango [-1, 1]:

Similar al escalado Min-Max pero ajustando los valores al rango de [-1, 1].

$$X_{normalized} = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \cdot (b - a) + a$$

```
scaler = MinMaxScaler(feature_range=(-1, 1))
data_normalized = scaler.fit_transform(data)
```

Variables numéricas: transformaciones

Transformación logarítmica. La empleamos cuando tenemos datos muy dispersos para que los valores de la variable queden “más juntos”. Ej.: Variable saldo en cuenta: la mayoría de los clientes tienen saldos de menos de 3.000€ y hay unos pocos com más de 500.000€.

```
df['log_values'] = np.log(df['values'])
```

Variables numéricas: discretización

Discretización: Proceso de transformar variables continuas en variables discretas mediante la creación de un conjunto de intervalos contiguos que abarcan el rango de los valores de las variables.

Este proceso se hace para:

- Manejar outliers
- Mejorar la distribución de variables: Esto es cuando tenemos la mayoría de los valores concentrados en un rango pequeño y algunos valores muy separados.

Variables numéricas: discretización→ KBinsDiscretizer

```
enc = KBinsDiscretizer(n_bins=10, encode='onehot', strategy='uniform')
X_binned =enc.fit_transform(X)
```

Encode

'onehot' (one-hot encoding): Cada bin se representa como una variable binaria (columna) que toma el valor 1 si el valor original cae dentro del bin, y 0 en caso contrario. Esto puede aumentar el número de características en el conjunto de datos, pero es útil para modelos que no manejan bien variables ordinales.
'ordinal' (ordinal encoding): Cada bin se representa como un valor entero que indica el orden del bin. Por ejemplo, si tienes tres bins, los valores serán 0, 1 y 2. Esto preserva el orden de los bins.
'onehot-dense' (one-hot encoding, formato denso): Similar a 'onehot', pero la matriz resultante es devuelta en un formato denso en lugar de esparcido (sparse).

Strategy

'uniform': Divide el rango de los valores de la característica en bins de igual tamaño.
'quantile': Divide los valores de la característica en bins con un número igual (o casi igual) de puntos de datos en cada bin.
'kmeans': Utiliza el algoritmo de agrupamiento k-means para encontrar los límites de los bins. Los puntos de datos se agrupan en k clústeres y los límites de los bins se definen por los centroides de estos clústeres.

Variables numéricas: binarización

Binarización: es el proceso de convertir datos continuos en binarios (dos valores posibles: 0 ó 1) basados en un umbral o condición específica. Simplifica los datos, haciéndolos adecuados para algoritmos que requieren entrada binaria o para resaltar distinciones específicas en los datos.

```
# Crear un binarizador con un umbral especificado
threshold = 2.0
binarizer = Binarizer(threshold=threshold)

# Aplicar la binarización a los datos
X_binarized = binarizer.fit_transform(X)
```

Outliers

Definición:

Los outliers (o valores atípicos) son datos que se alejan significativamente de la mayoría de los valores en un conjunto de datos. En estadística, suelen indicar algo inusual, como un error en la medición, una variabilidad inesperada o un evento especial.

Pueden ser anomalías en los datos o errores y conviene tratarlos ya que a veces los modelos pueden tender a sobre adaptarse al outlier.

Existen varias maneras de detectarlos:

Percentiles

Las observaciones que están por debajo del percentil 5 (a veces el 1) o por encima del percentil 95 (a veces el 99) se consideran valores atípicos. Este método es apropiado para distribuciones simétricas y normales.

Rango intercuartílico (IQR)

Si Q1 es el primer cuartil y q3 es el tercer cuartil, entonces $IQR = Q3 - Q1$. Las observaciones que están por debajo de $Q1 - 1.5 \times IQR$ o por encima de $Q3 + 1.5 \times IQR$ se consideran valores atípicos. En un box plot, esto corresponde a los puntos de datos que caen fuera de los bigotes.

Z-score

El z-score de una observación representa su distancia respecto a la media, en términos de desviaciones estándar. Cualquier observación que esté a más de 3 desviaciones estándar de la media se considera un valor atípico.

Outliers: tratamiento

- Eliminar la observación, es decir, realizar una eliminación por lista.
- Conservar la observación tal cual.
- Reemplazar la observación utilizando uno de los mismos métodos de imputación que para los datos omitidos.

Conservando el valor actual también se puede aplicar alguna de las transformaciones que ya hemos visto: Scaling, normalization, discretización, transformación.

Variables de texto: Bag of Words (BoW)

Bag of Words:

Es una técnica de vectorización de textos, donde cada palabra se convierte en una variable que toma valores 1 o 0 dependiendo de si está presente en el texto o no.

```
from sklearn.feature_extraction.text import CountVectorizer

# Documentos de ejemplo
documents = [
    "El gato está en el tejado",
    "El perro está en el jardín",
    "El gato y el perro son amigos"
]

# Crear un vectorizador Bag of Words
vectorizer = CountVectorizer()

# Ajustar y transformar los documentos
bow_matrix = vectorizer.fit_transform(documents)

# Obtener los nombres de las características (palabras)
feature_names = vectorizer.get_feature_names_out()

# Convertir la matriz Bag of Words en un DataFrame de pandas para mayor claridad
import pandas as pd
bow_df = pd.DataFrame(bow_matrix.toarray(), columns=feature_names)

print(bow_df)
```

Variables de texto: TF-IDF

La transformación TF-IDF (Term Frequency-Inverse Document Frequency) convierte un conjunto de documentos de texto en una matriz de valores numéricos que representan la importancia de cada término en cada documento. Cada valor en la matriz TF-IDF se calcula de la siguiente manera:

1. **TF (Term Frequency):**
   La frecuencia de un término en un documento. Es decir, el número de veces que un término aparece en un documento dividido por el número total de términos en ese documento.

2. **IDF (Inverse Document Frequency):**
   Una medida de cuán único es un término en el conjunto de documentos. Se calcula como el logaritmo del número total de documentos dividido por el número de documentos que contienen el término.

$$TFIDF(t,d) = TF(t,d) \times log(\frac{N}{DF(t)})$$

Donde:

- TF(t,d) es la frecuencia del término t en el documento d. Esto es el cociente entre el número de veces que aparece t en el documento y el número total de palabras del documento d.
- N es el número total de documentos.
- DF(t) es el número de documentos que contienen el término t.

¿Qué es un pipeline?

¿Qué es un pipeline?

Un pipeline en Python se refiere a una secuencia de operaciones o transformaciones que se aplican a los datos de manera encadenada. Es como una tubería donde los datos fluyen de una etapa a otra, siendo procesados y transformados en cada paso.

¿Por qué usar pipelines?

- **Legibilidad:** El código se vuelve más conciso y fácil de entender.
- **Reutilización:** Puedes crear pipelines personalizados y reutilizarlos en diferentes partes de tu código.
- **Eficiencia:** Al encadenar operaciones, puedes evitar crear variables intermedias innecesarias.
- **Mantenimiento:** Es más fácil modificar o agregar nuevas etapas a un pipeline.

```
pipeline = Pipeline([
    ('MyHotEncoder', FunctionTransformer(MyHotEncoder)),        # Paso 1: gestiona variables categoricas
    ('quitar_nulos', FunctionTransformer(quitar_nulos)),        # Paso 2: quita registros con nulos
    ('normalizar', FunctionTransformer(normalizar))             # Paso 3: Normalizar
])
X_transf = pipeline.fit_transform(X)
```